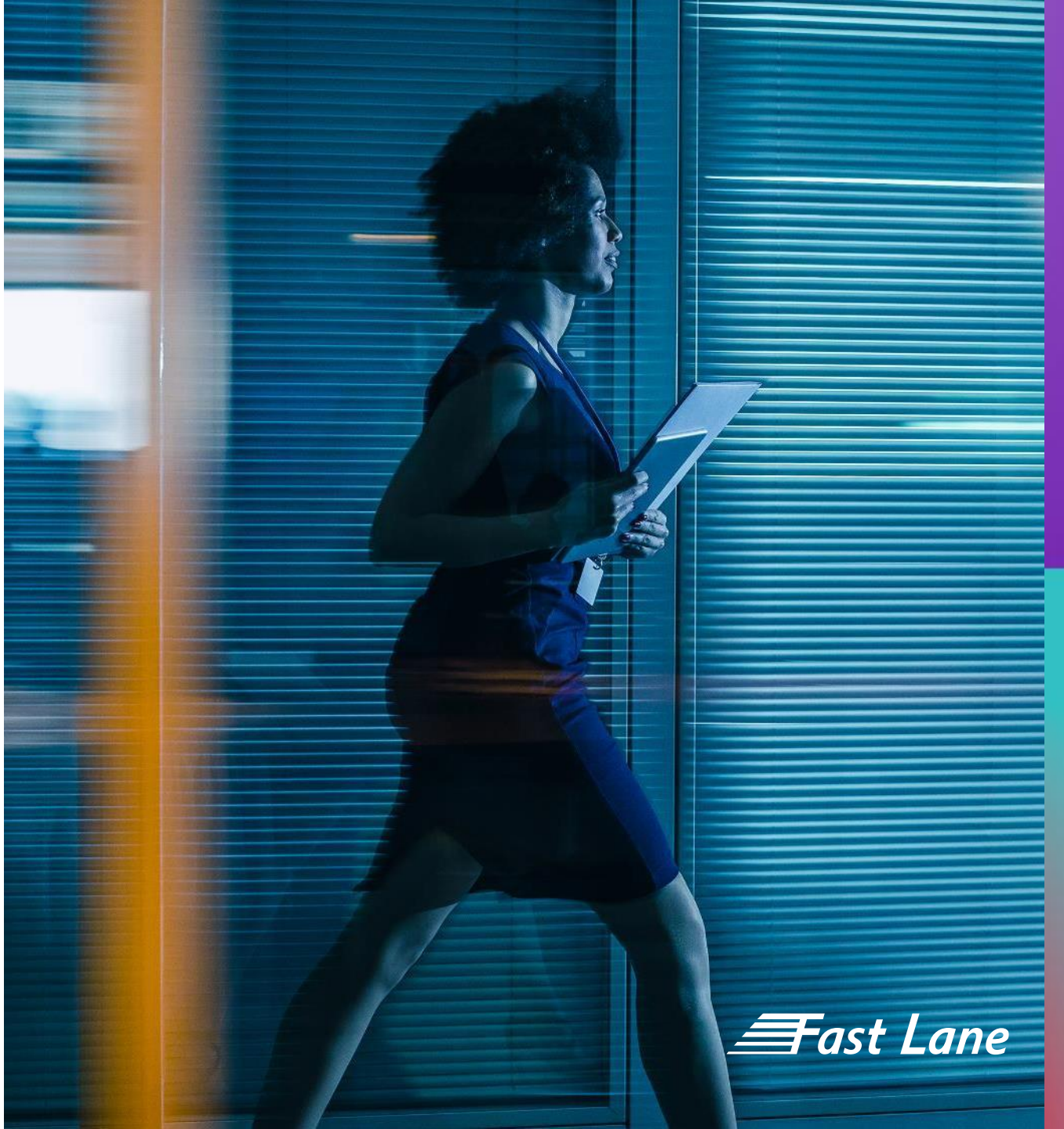
Fast Lane Group

Worldwide Education & Professional Services

Microsoft Solutions Partner

Microsoft Cloud Training Services



www.flane.de | © 2026 Fast Lane Institute for Knowledge Transfer GmbH

— AGENTIC AI: FROM STRATEGY TO **PRODUCTION.**

Alexander Kastil

Full stack, from enterprise software to agentic AI. Since 1996

AI at Work
Value discovery, KPI mapping and roadmap

Enterprise AI
Governed multi-agent systems on Microsoft Copilot and Foundry

Agentic Software Engineering
Harness architecture for Claude Code and GitHub Copilot

Team Enablement
AI driven content production and delivery

integrations.at
alexander.kastil@integrations.at

github.com/alexander-kastil
linkedin.com/in/alexander-kastil

INTEGRATIONS

AGENTIC AI CONSULTING · VIENNA



Microsoft CERTIFIED — AGENTIC AI BUSINESS SOLUTIONS ARCHITECT — ★★★

Microsoft CERTIFIED — AZURE AI APP AND AGENT DEVELOPER — ASSOCIATE ★★

Microsoft CERTIFIED — DEVOPS ENGINEER — EXPERT ★★★

SCAN TO CONNECT
integrations.at

MODULE 5

Advanced Copilot Studio Agents

Agents that run on their own and work with other agents

Automate Business Processes with Copilot Studio and Copilot Cowork



01

Autonomous Agents & Event Triggers

Event triggers, unattended runs and a shared returns mailbox

02

Deep Reasoning Models

The reason keyword on the one step that weighs the rules

03

Generative Orchestration & Multi-Agent Routing

The planner, child agents and connected agents

04

Microsoft Foundry Agents: Core Concepts

Prompt agents, versions and the stable endpoint

05

Foundry Agents in Copilot Studio

A Foundry specialist behind a Standard agent

06

Voice Agents: IVR & Post-Call

Optional: speech and keypad menus, then work after the call

07

Test and Evaluate Standard Agents

Optional: test sets, graders and one change per run

Advanced Copilot Studio Agents

Agents that run on their own and work with other agents

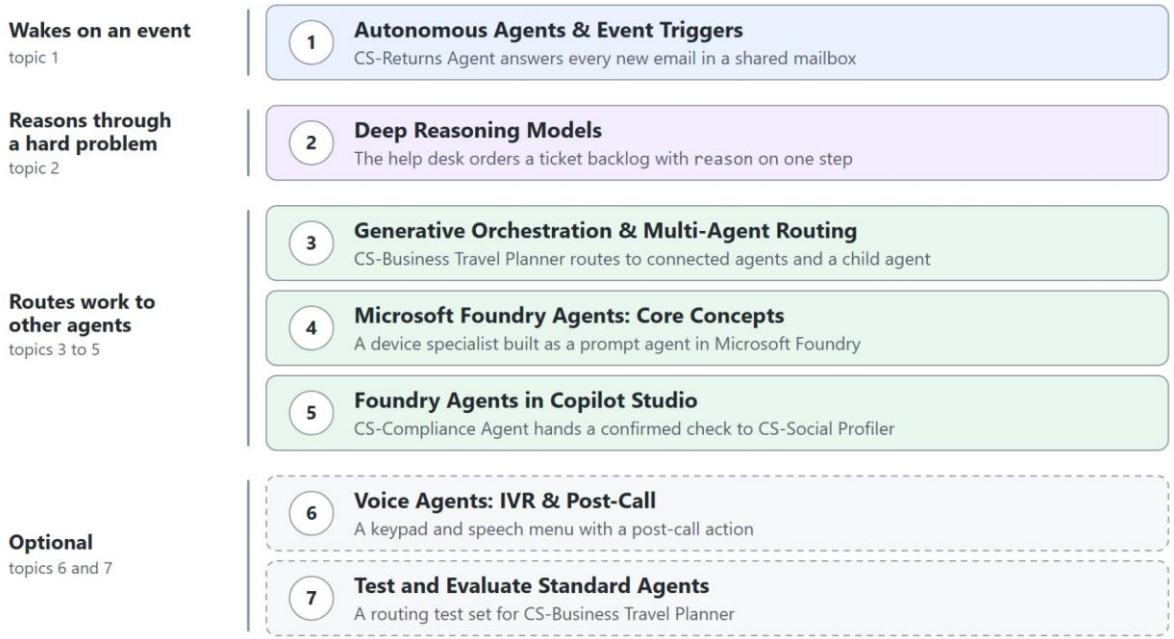
- **On its own**: an event starts the agent, with nobody in the chat
- **With other agents**: one front door routes work to child, connected and Microsoft Foundry agents
- **Harder problems**: deep reasoning for the one step that has to weigh several rules



Overview

Seven topics in delivery order

- Start with an agent that **wakes on an event** and answers by mail
- Give one hard step a **deep reasoning** model with the `reason` keyword
- Route work to **child and connected agents** through generative orchestration
- Bring in **Microsoft Foundry** agents that developers build and own
- Optional: **voice** menus on the phone and **test sets** that grade the agent



01

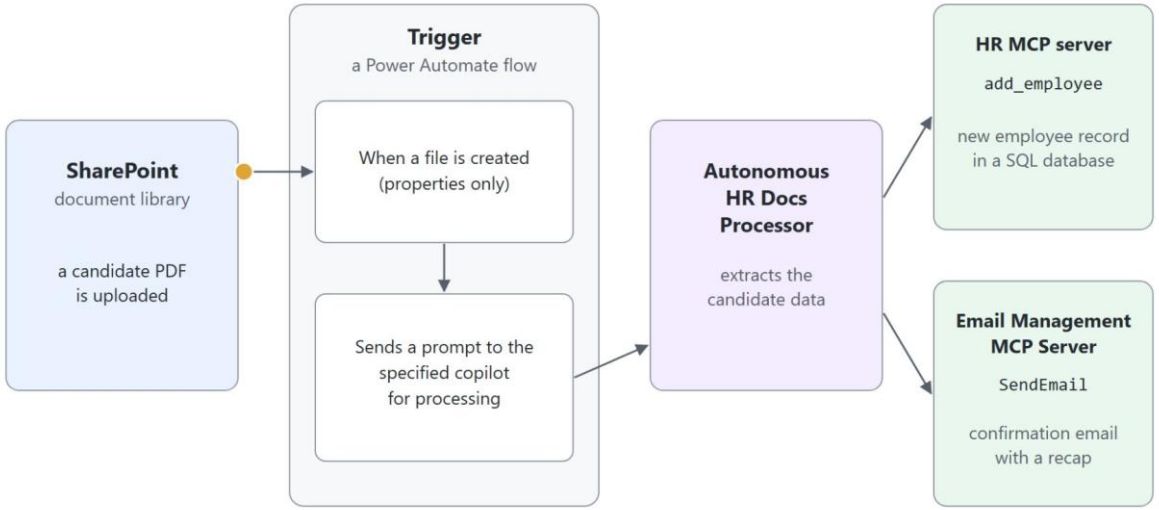
Autonomous Agents & Event Triggers

Event triggers, unattended runs and a shared returns mailbox

Autonomous agents and triggers

A trigger is a Power Automate flow that prompts the agent

- An **event trigger** fires when something happens in another system, not when a user types
- It sends the agent a **payload** with the event's data, and the instructions decide which tools run
- Behind every trigger sits a small **Power Automate flow** that sends the agent a prompt
- In the lab, a resume PDF in SharePoint becomes an employee record through the **HR MCP server**



CS-Returns Agent on a shared mailbox

Every new return request gets an answer by mail

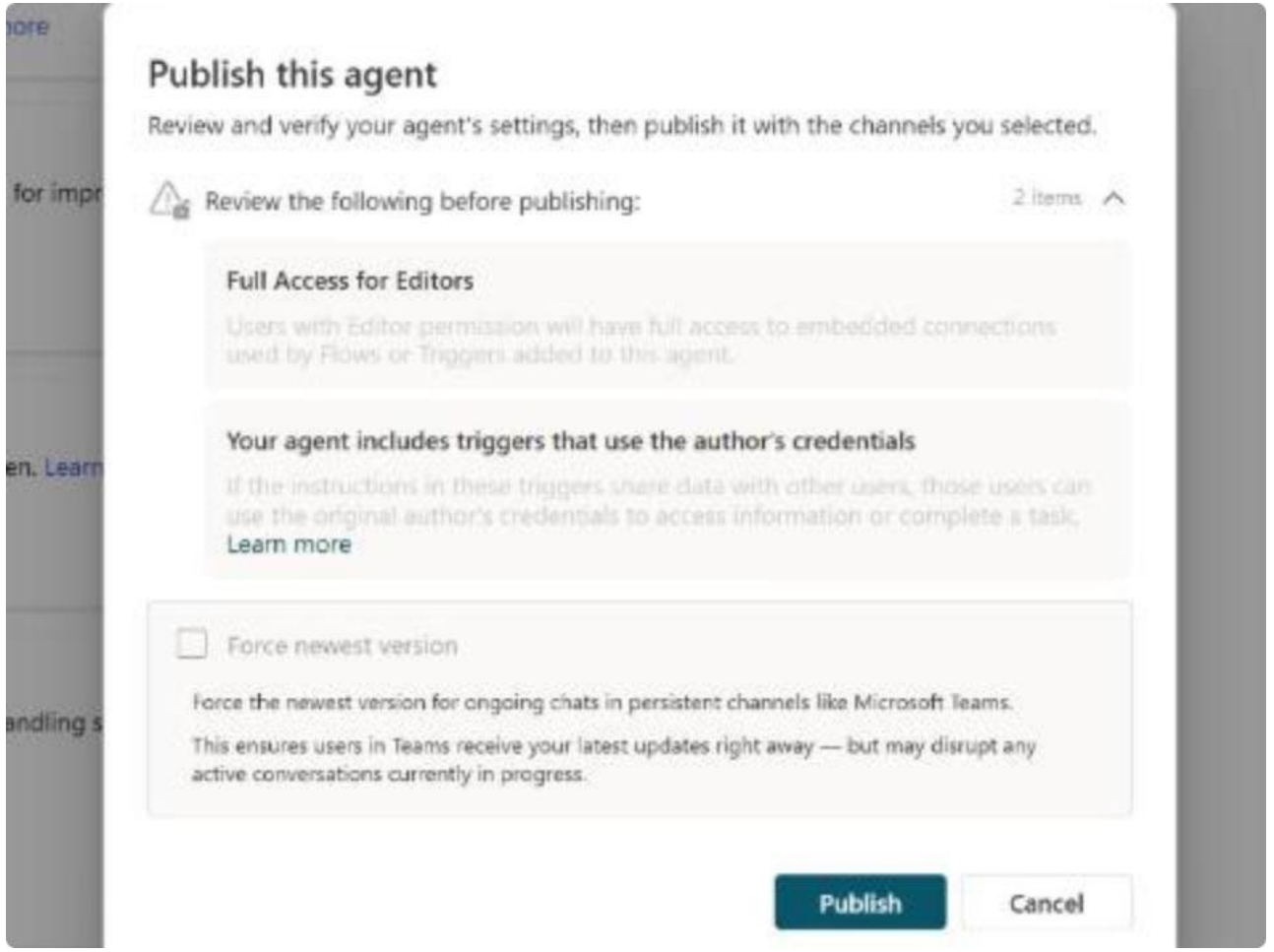
- **Instructions** limit the agent to returns and set the reply format: an email with `<p>` and `<br>` tags
- **returns-policy.docx** from SharePoint is its only knowledge source
- **CS-Work IQ Mail** runs on maker-provided credentials, because no user is there to sign in
- One sentence names the tool: "Send that email to the customer with CS-Work IQ Mail."
- **CS-When a new email arrives in a shared mailbox** hands every new email to the agent



Testing and publishing the trigger

Replay a recorded email first, then let the agent work alone

- Before you publish, the trigger records each email but its flow run stops at the agent call
- The test icon replays a recorded email: **Trigger detected**, then **Email sent** citing returns-policy.docx
- **Publish** lists two items to review: Full Access for Editors and triggers that use the author's credentials
- After publishing, every run appears as an **Automated** row on the Activity page



02

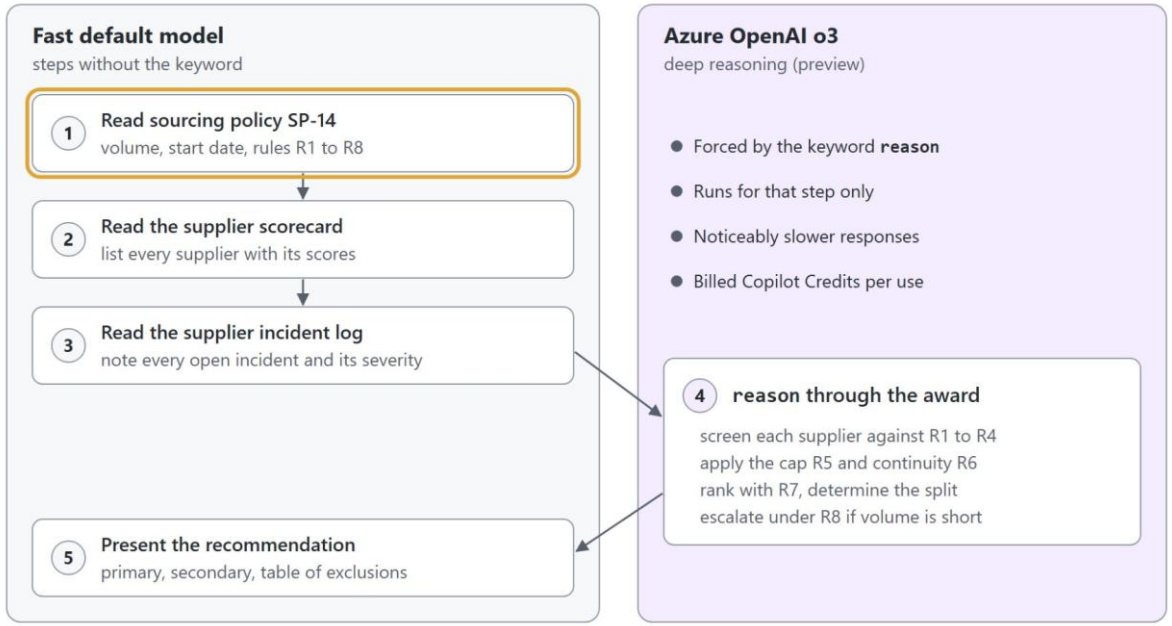
Deep Reasoning Models

The reason keyword on the one step that weighs the rules

Deep reasoning and the reason keyword

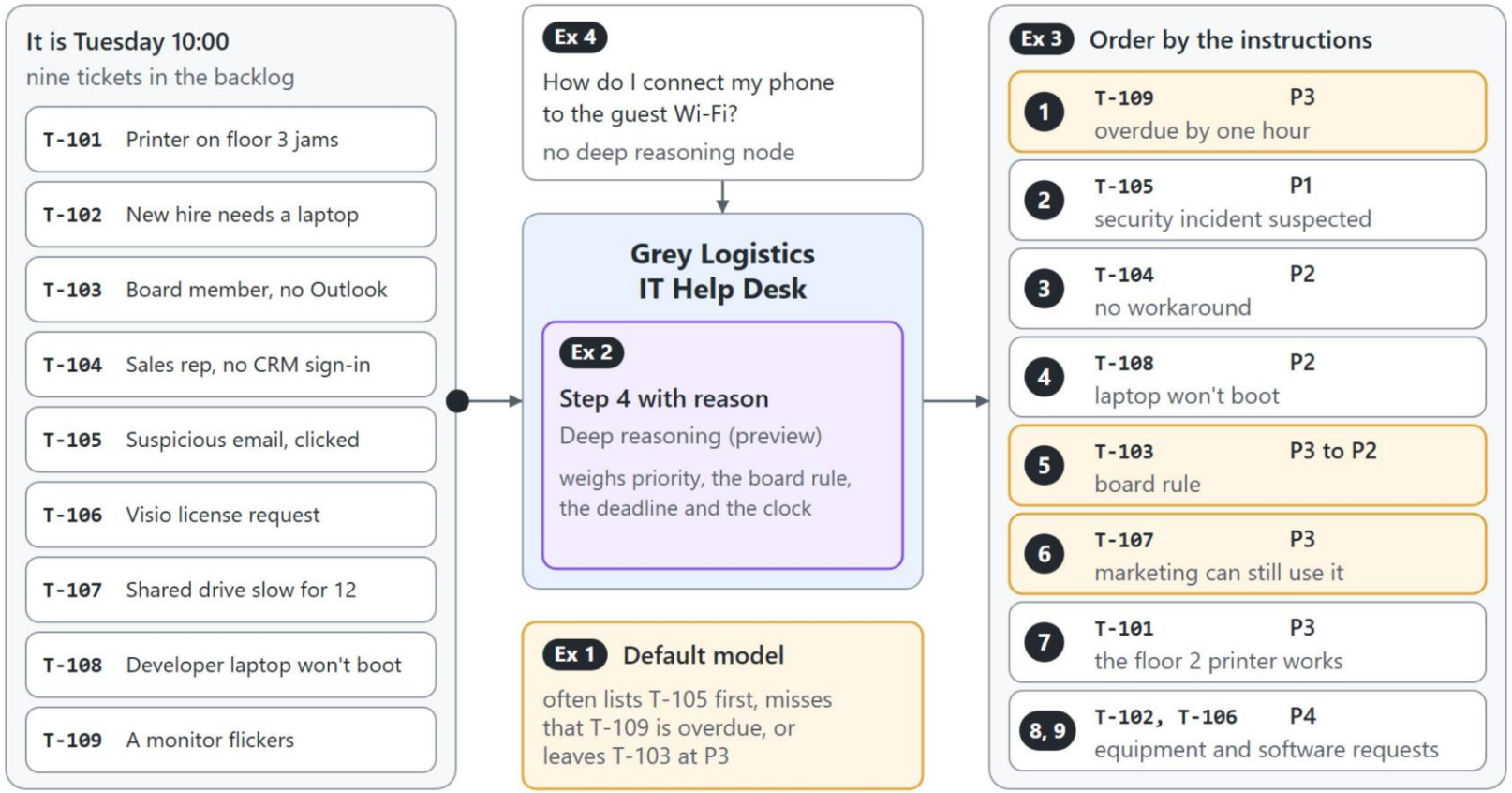
Only the step that carries the keyword runs on the reasoning model

- **Deep reasoning (preview)** is a switch in the agent's Settings and needs generative orchestration
- The keyword `reason` in an instruction step forces the reasoning model for **that step only**
- Learn's deep reasoning article names **Azure OpenAI o3** for the reasoning steps
- The **activity map** shows a deep reasoning node wherever the model ran



Ordering a ticket backlog

The same nine tickets on the default model and with reason on step 4



- With `reason` on the ranking step, T-109 comes first as overdue, T-103 rises to P2 and T-107 stays at P3

03

Generative Orchestration & Multi-Agent Routing

The planner, child agents and connected agents

Generative orchestration

How the planner turns a request into a multi-step plan

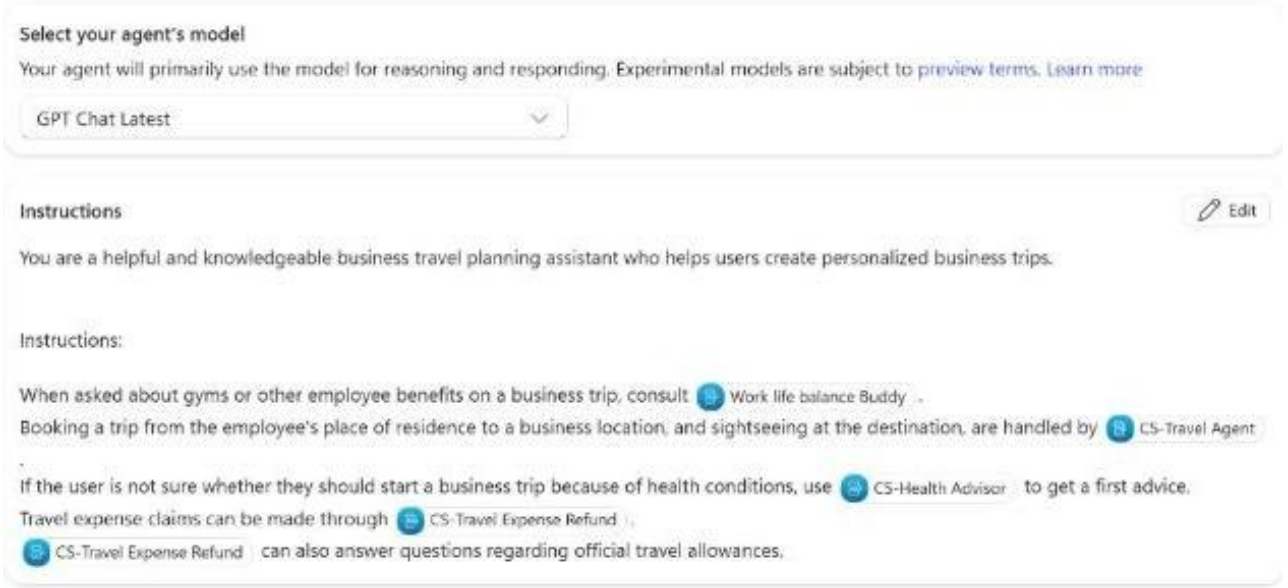
- The planner reads a request and picks **topics, tools, knowledge and other agents** to answer it
- The **description** weighs most in that choice, then the name
- One plan can chain **several steps** and writes one reply from what they returned
- Event triggers, deep reasoning and other agents all need **generative orchestration**



One front door for the travel specialists

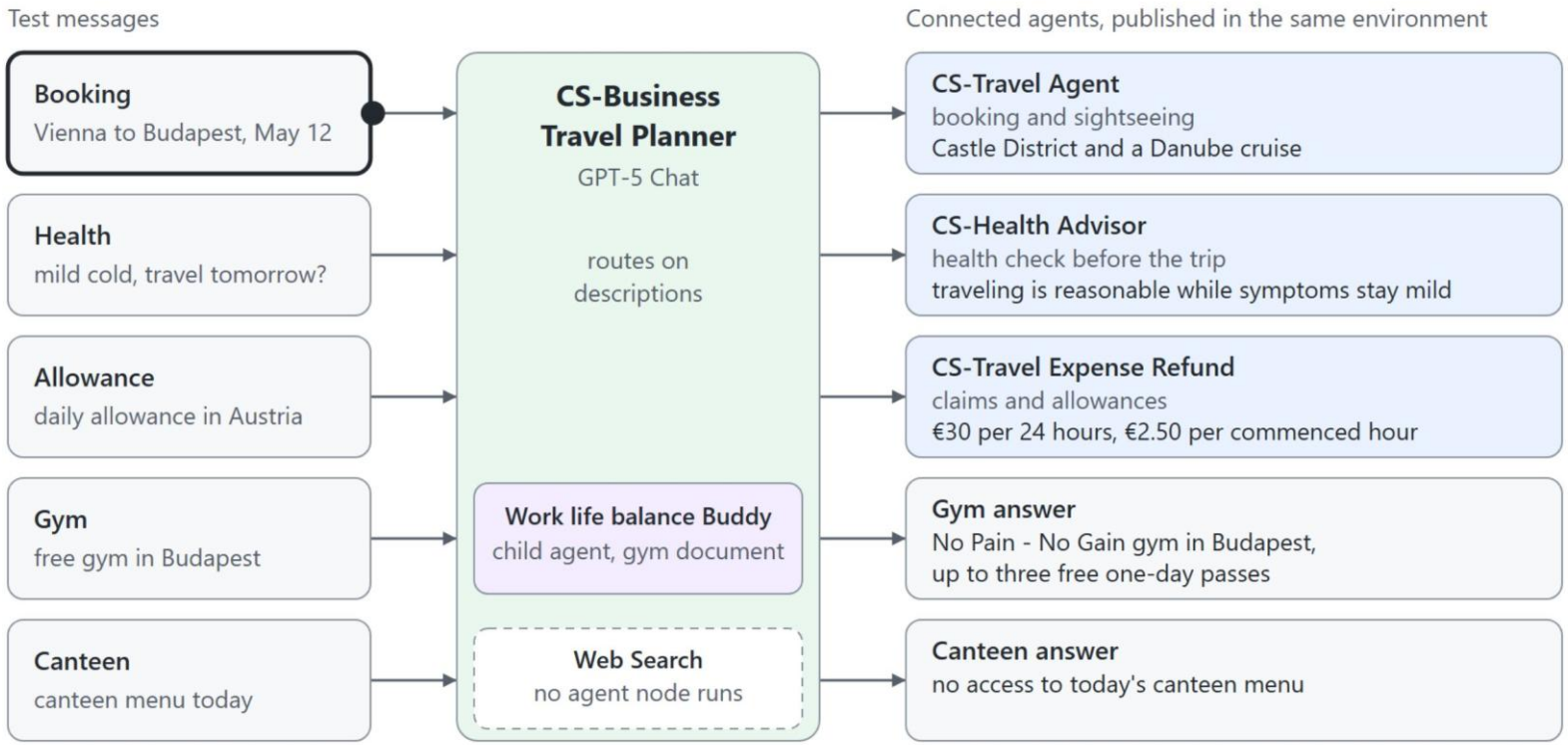
CS-Business Travel Planner names a specialist for every request

- Three **connected agents**: CS-Travel Agent, CS-Health Advisor and CS-Travel Expense Refund
- One **child agent**, Work life balance Buddy, answers from a gym document only it can see
- Each description says what the specialist covers and what it **does not handle**
- Typing `/` on an agent name turns it into a **reference**, shown as a chip



Proving every route

Five test messages, each answered by exactly one agent or by none



- With the activity map on, each message shows the agent node that answered it, and the canteen question shows none

04

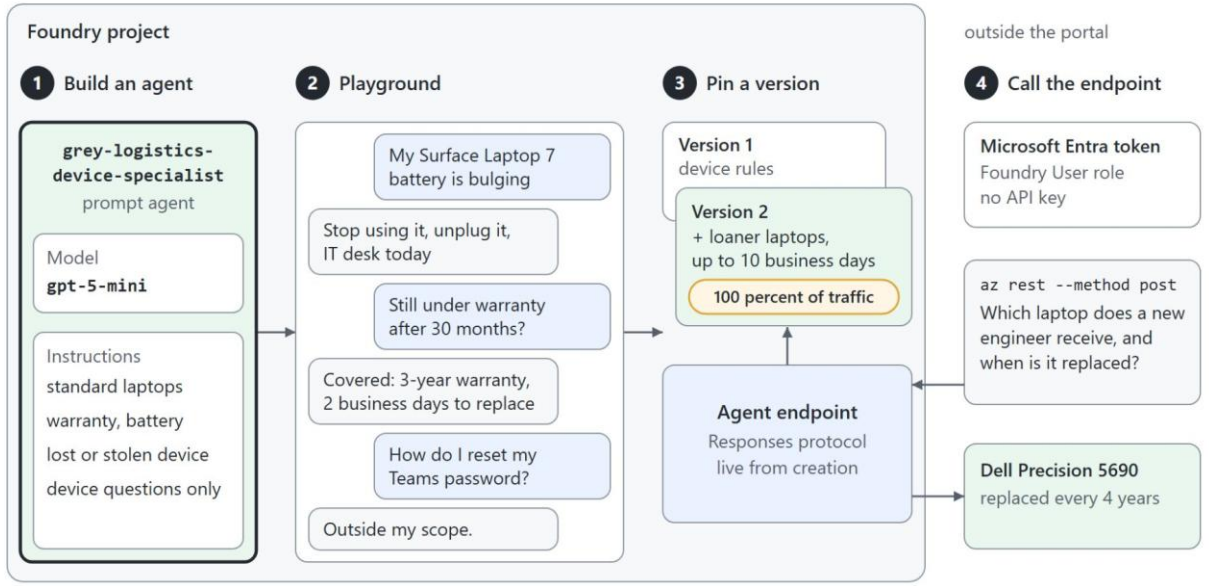
Microsoft Foundry Agents: Core Concepts

Prompt agents, versions and the stable endpoint

A prompt agent in Microsoft Foundry

Model, instructions and tools behind a stable endpoint

- A **Foundry project** groups agents, files and tools inside an Azure subscription
- A **prompt agent** is configuration only: a model, instructions and optional tools
- Every saved change creates an **immutable version**; you pin the one the endpoint serves
- The endpoint takes a **Microsoft Entra** sign-in; API keys aren't supported



Foundry agents and Copilot Studio agents

Who builds, where it runs and how it reaches users

The maker keeps the conversation, the channels and the routing; the developer team keeps the specialist's model, tools and versions



05

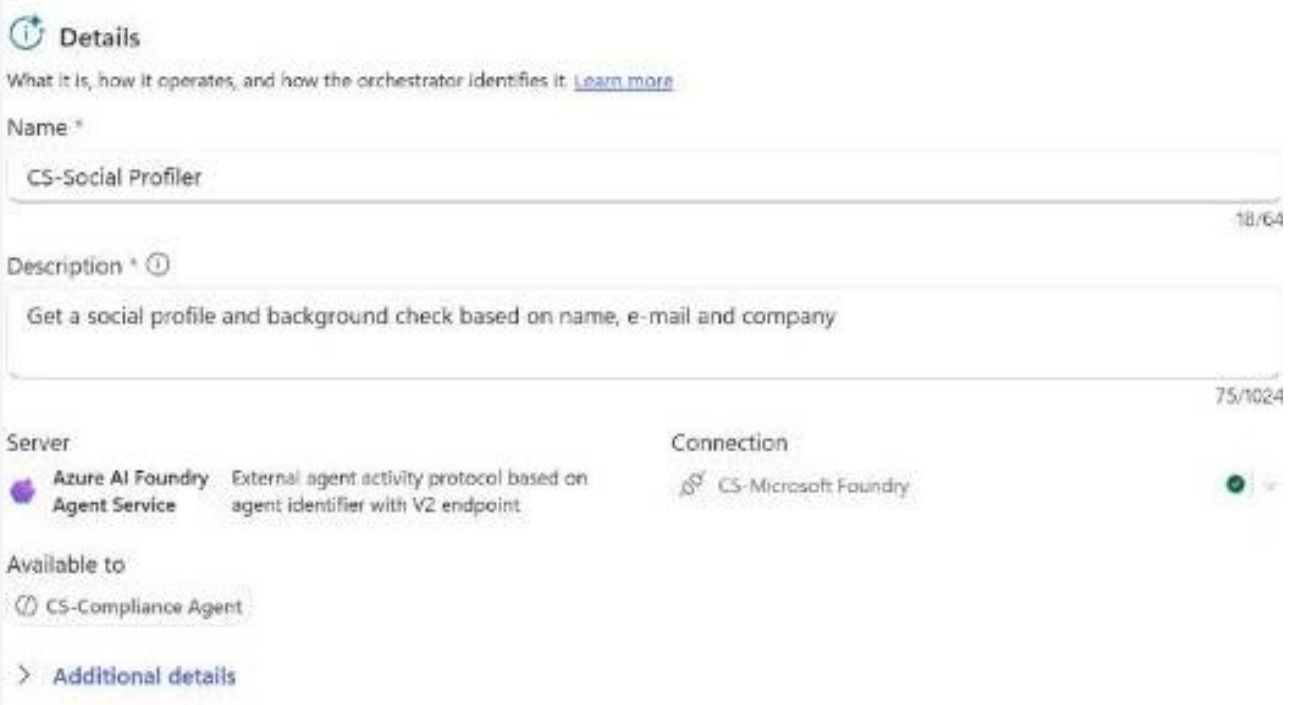
Foundry Agents in Copilot Studio

A Foundry specialist behind a Standard agent

Connecting a Foundry agent

The Activity Protocol on the Foundry side, a connection and an Agent Id in Copilot Studio

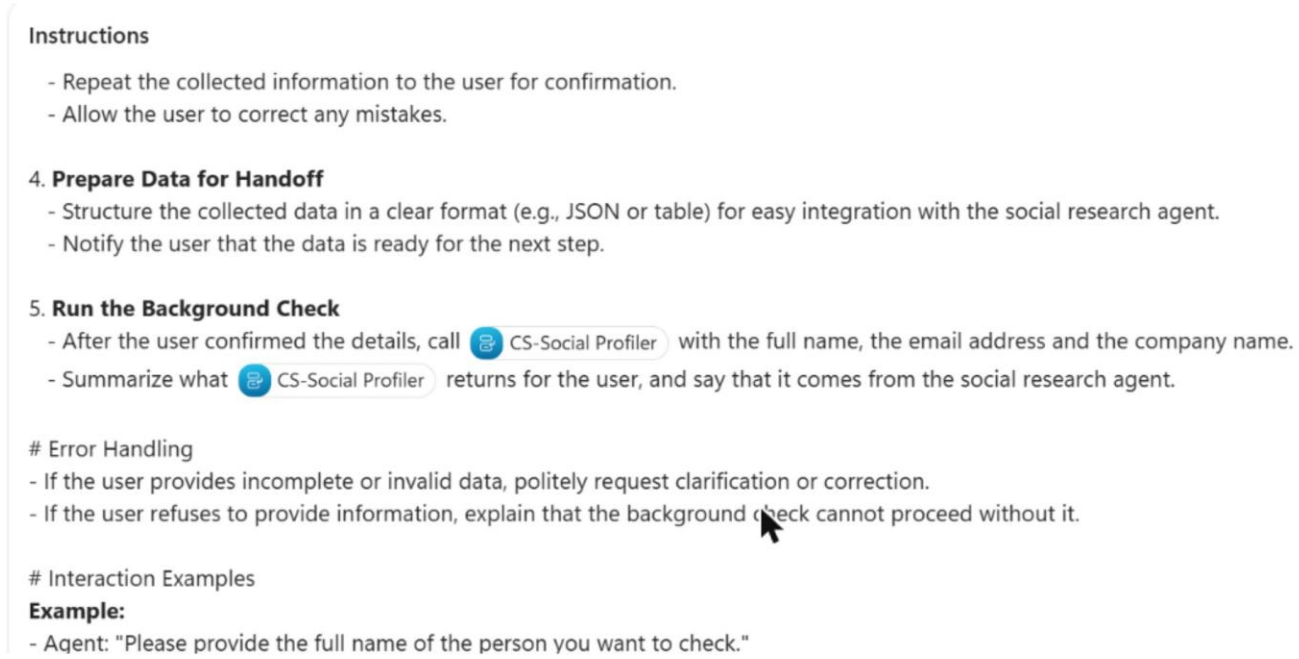
```
{
  "agent_endpoint": {
    "protocol_configuration": {
      "responses": {},
      "activity": {},
      "a2a": {}
    },
    "authorization_schemes": [
      { "type": "Entra" },
      { "type": "BotServiceRbac" }
    ]
  }
}
```



Collect, confirm, then hand off

CS-Compliance Agent calls CS-Social Profiler only after the user confirms

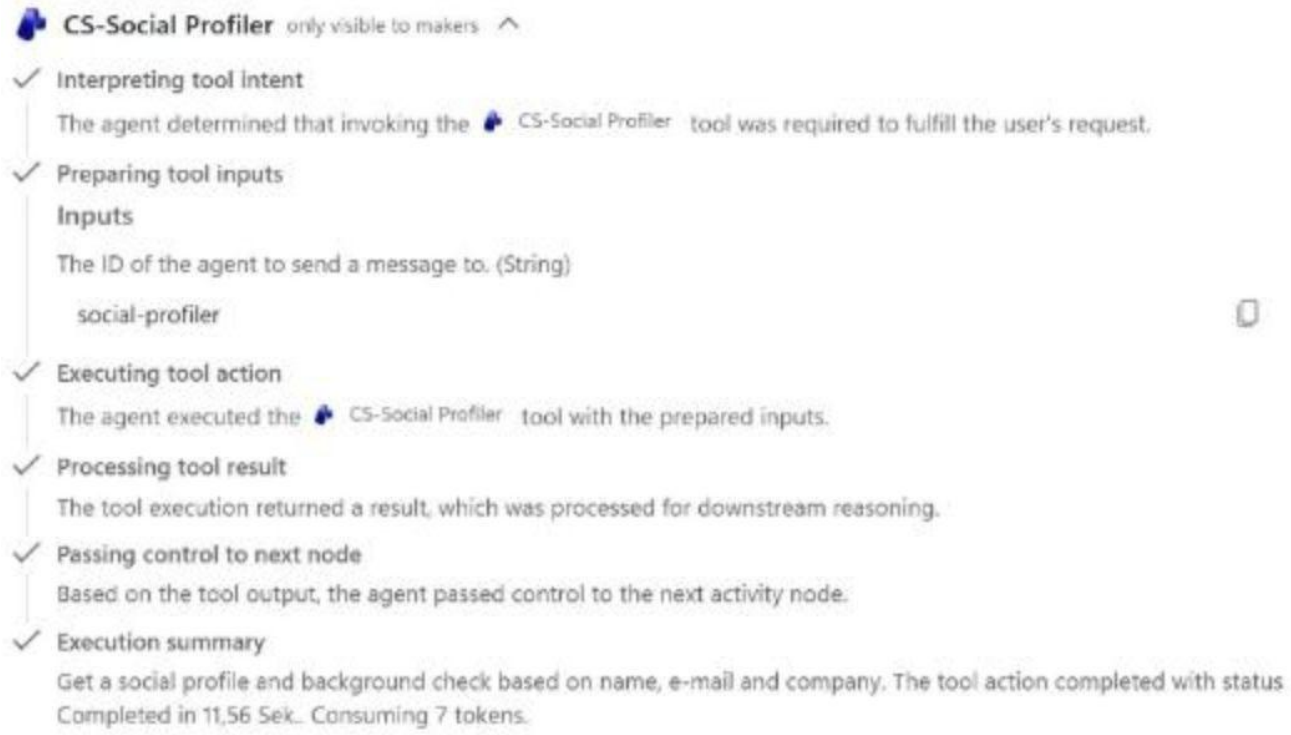
- **Claude Sonnet 4.6** runs CS-Compliance Agent, chosen for following long, ordered instructions
- Numbered steps explain the check, collect name, email and company, then **confirm** them
- New **step 5** calls CS-Social Profiler only after the user confirmed the details
- **Error handling** asks again for invalid data, so a broken email address stops the check



The Foundry call in the activity map

CS-Social Profiler runs once the details are confirmed

- The **CS-Social Profiler** node appears only after "Yes, that's correct."
- Its input carries the Agent Id **social-profiler**
- The reply comes back in five sections, from **Person** to **Notes for the compliance check**
- Each hop runs its own model, so expect more **latency** than for a tool



06

Voice Agents: IVR & Post-Call

Optional: speech and keypad menus, then work after the call

Voice agents

A phone menu that answers to speech and the keypad

- **Basic** voice agents use classic orchestration; **real-time** voice agents use generative orchestration
- DTMF keys map the keypad to the menu: 1 for Account, 2 for Device, 0 for a person
- The **Speech & DTMF** chat mode tests a call in text, and /DTMFkey 1 presses a key
- A post-call topic runs on **End of conversation** and filters on Conversation.EndReason



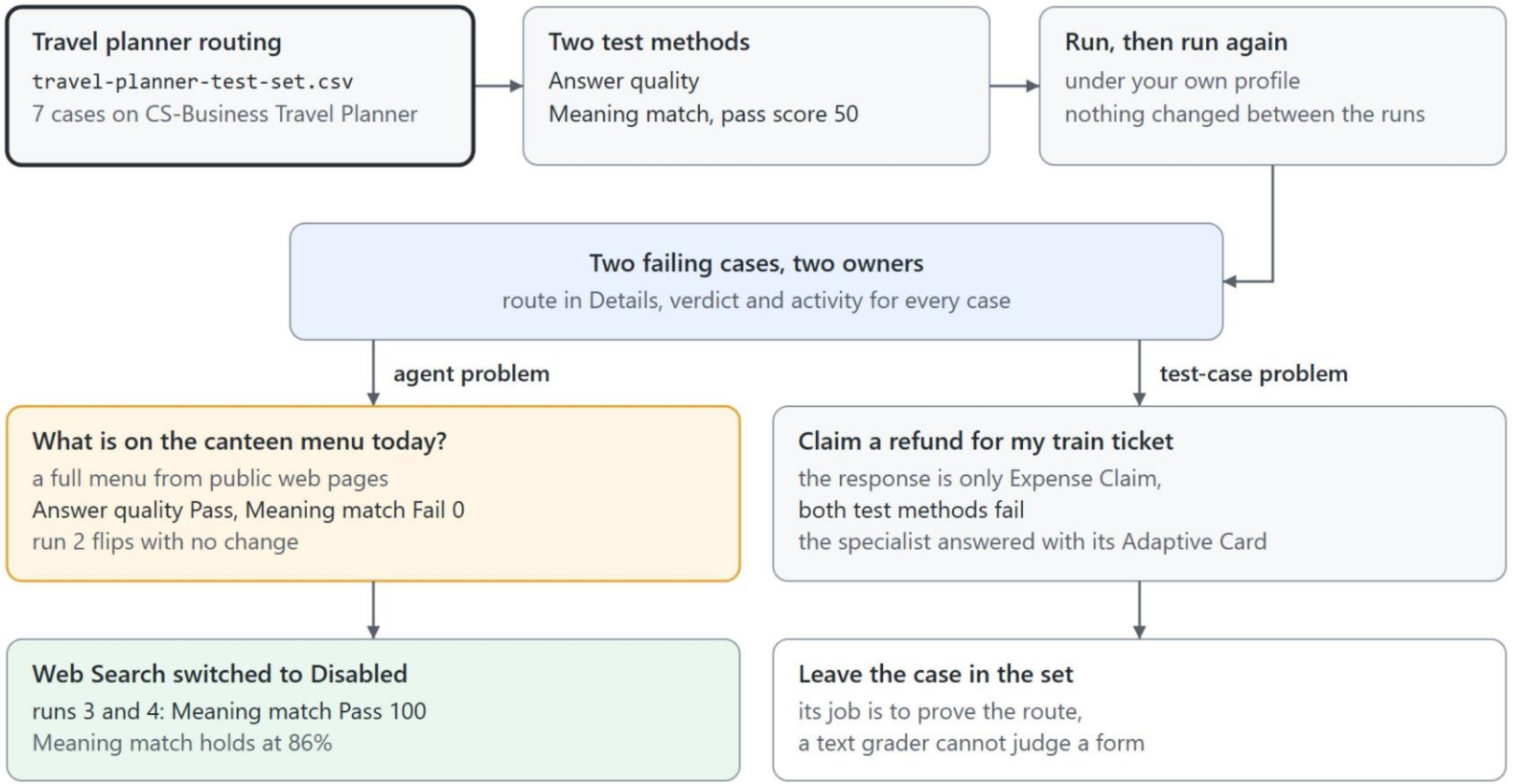
07

Test and Evaluate Standard Agents

Optional: test sets, graders and one change per run

A routing test set for the travel planner

Seven cases, two test methods and two failing cases with different owners

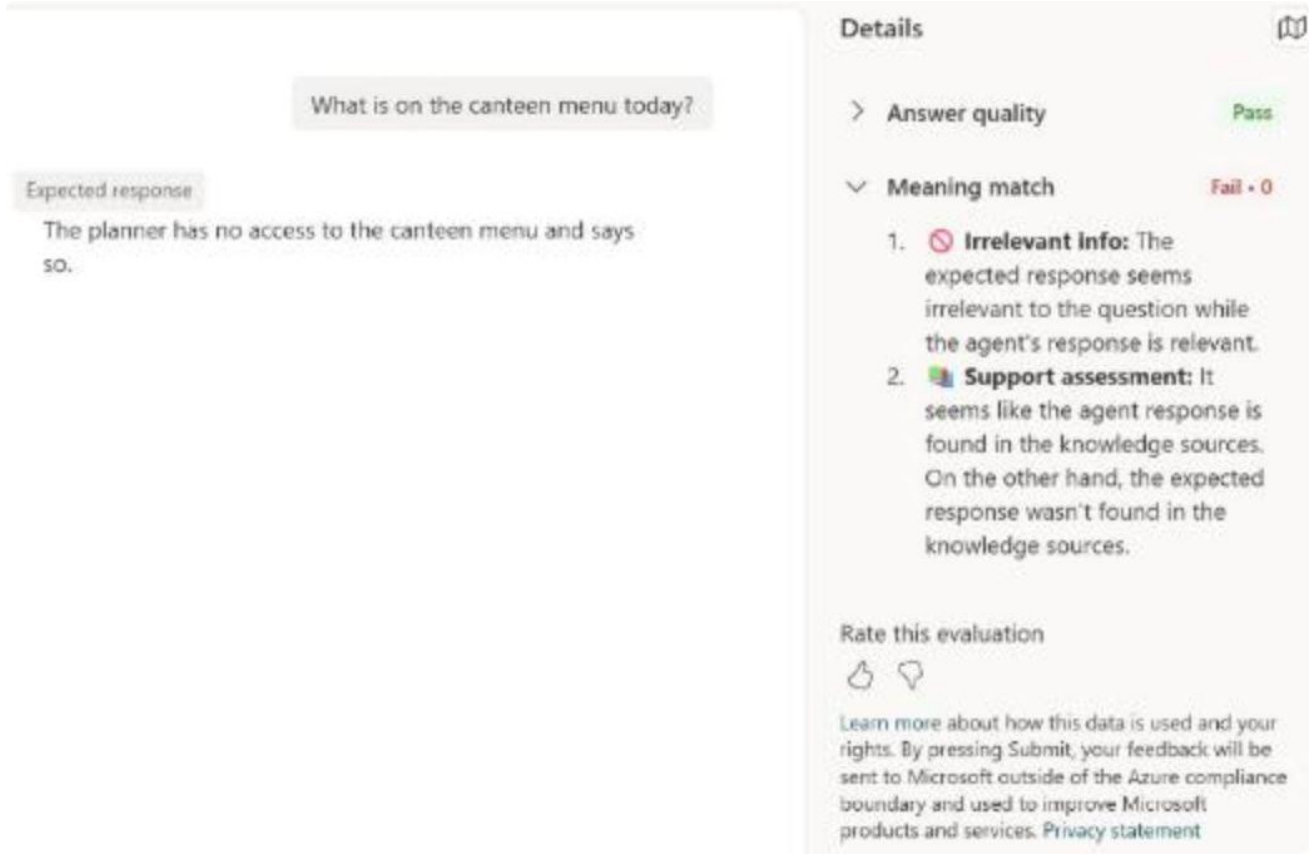


The baseline run scored Answer quality 86% and Meaning match 71%, and its two failures had two different owners

What Answer quality misses

A confident answer from the wrong source can pass it

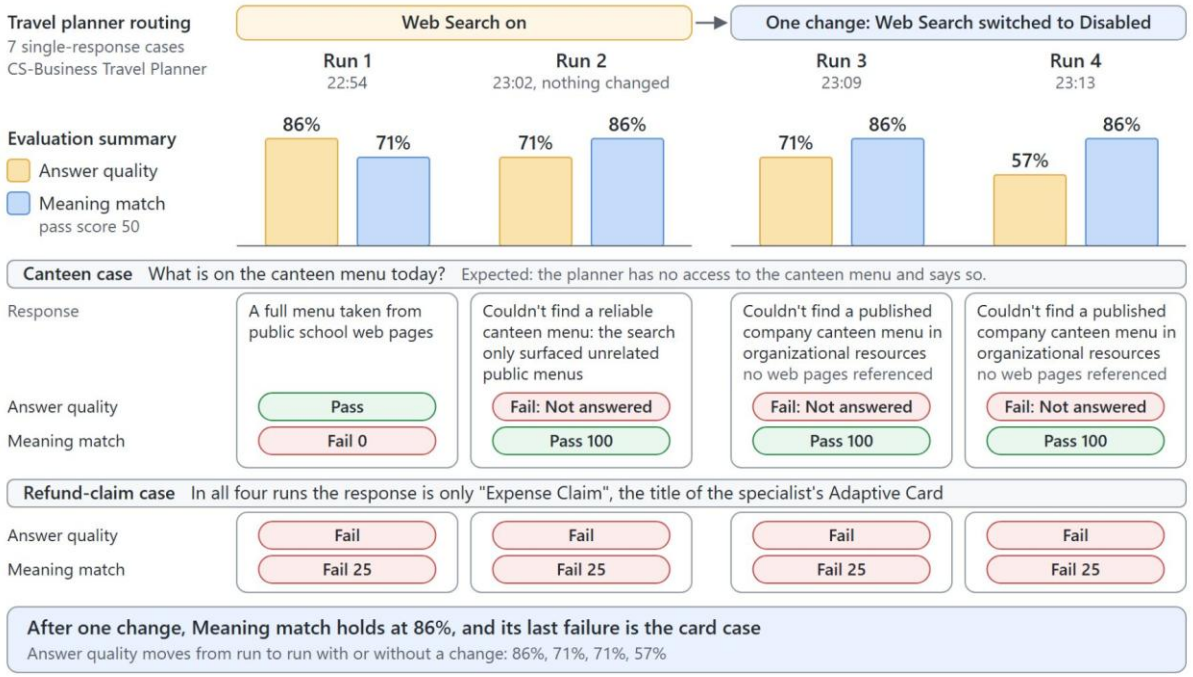
- **Answer quality** judges the reply alone and never reads the expected response
- The planner answered the canteen question with a full menu from public school web pages
- Answer quality passed it; **Meaning match** scored it 0 against "has no access"
- Pair Answer quality with a **comparing method** on cases with one right answer



Comparing runs before and after a change

One change, Web Search switched to Disabled, fixed the canteen case

- Run the **unchanged agent twice** first: 86% and 71%, then 71% and 86%
- The canteen case flipped on its own, and its activity listed **public web pages**
- One change: **Web Search** switched to Disabled on the planner
- **Meaning match** held at 86% in both later runs, while Answer quality moved to 71% and 57%



Takeaways

What to carry into the Microsoft Copilot harness agents module

- An unattended run has nobody to ask: the **instructions** carry the whole job, the tools use maker credentials
- Put reason only on the step that has to **weigh several rules**
- Route on **descriptions** that each claim one job, and prove every route in the activity map
- Let developers own a **Foundry specialist** and connect it over the Activity Protocol
- Gate changes on a **test set** with a comparing method, one change per run

